

SENAI BETIM MARIA MADALENA NOGUEIRA

HENRY GABRIEL DOS SANTOS ROSA

JOÃO ELIS RODRIGUES DE ARAÚJO

LEANDRO GONÇALVES STANCIOLA

SAMUEL GOMES DO NASCIMENTO

DESENVOLVIMENTO DE SISTEMAS

Trabalho de Conclusão de Curso

BETIM

2026

HENRY GABRIEL DOS SANTOS ROSA
JOÃO ELIS RODRIGUES DE ARAÚJO
LEANDRO GONÇALVES STANCIOLA
SAMUEL GOMES DO NASCIMENTO

DESENVOLVIMENTO DE SISTEMAS

Trabalho de Conclusão de Curso

Trabalho de Conclusão de Curso apresentado ao SENAI BETIM MARIA
MADALENA
NOGUEIRA como requisito para aprovação no curso tecnólogo Desenvolvimento de
Sistemas.

BETIM

2026

SUMÁRIO

1. INTRODUÇÃO	5
2. DESENVOLVIMENTO	5
2.1 OBJETIVO DO SISTEMA	5
2.2 TECNOLOGIAS UTILIZADAS	6
2.3 REQUISITOS DO SISTEMA.....	8
2.4 ESTRUTURA DO PROJETO (BACK-END)	9
2.4.1 CAMADA CONTROLLER	9
2.4.2 CAMADA DTO.....	10
2.4.3 CAMADA EXCEPTION	10
2.4.4 CAMADA MODEL	11
2.4.5 CAMADA REPOSITORY	11
2.4.6 CAMADA SERVICE	11
2.4.7 CAMADA CONFIG	12
2.5 BANCO DE DADOS.....	12
2.5.1 TABELA PERFIL	14
2.5.2 TABELA USUÁRIO	14
2.5.3 TABELA STATUS_VERSAO_PRODUTO.....	14
2.5.4 TABELA PRODUTO.....	15
2.5.5 TABELA ESTOQUE	15
2.5.6 TABELA RECEITA	16
2.5.7 TABELA VERSAO_RECEITA.....	16
2.5.8 TABELA TESTE	17
2.5.9 TABELA HISTÓRICO.....	17
2.5.10 RELACIONAMENTOS	18
2.6 ESTRUTURA DA API.....	19
2.6.1 MÉTODOS HTTP	20
2.6.2 USUARIO CONTROLLER	21
2.6.3 PERFIL CONTROLLER.....	21
2.6.4 PRODUTO CONTROLLER.....	22
2.6.5 StatusProduto CONTROLLER.....	22
2.6.6 ESTOQUE CONTROLLER.....	23

2.6.7 RECEITA CONTROLLER	24
2.6.8 VersaoReceita CONTROLLER.....	24
2.6.9 TESTE CONTROLER	25
2.6.10 HISTORICO CONTROLLER	25
2.6.11 AUTH CONTROLLER.....	26
2.7 TRATAMENTO DE EXCEÇÕES	26
2.7.1 GlobalExceptionHandler	26
2.7.2 ErrorResponse	27
2.7.3 EXCEÇÕES PERSONALIZADAS	27
2.7.4 FLUXO DO TRATAMENTO DE EXCEÇÕES	28
2.7.5 EXEMPLO DE RESPOSTA	29
3. MANUAL DE INSTALAÇÃO E EXECUÇÃO (BACKEND)	30

1. INTRODUÇÃO

O presente manual técnico tem como objetivo orientar a instalação, configuração e execução do sistema OneMonth, desenvolvido como Trabalho de Conclusão de Curso. Este documento reúne as informações necessárias para que desenvolvedores ou avaliadores possam preparar o ambiente, configurar o banco de dados, executar o back-end e o front-end e realizar testes básicos da aplicação.

Além das instruções de instalação, o documento apresenta as tecnologias utilizadas no desenvolvimento do projeto, a organização da estrutura do sistema e orientações para solucionar os problemas mais comuns durante a execução. O objetivo é facilitar a utilização da aplicação e servir como referência para futuras manutenções e atualizações.

2. DESENVOLVIMENTO

2.1 OBJETIVO DO SISTEMA

O sistema **OneMonth** foi desenvolvido com o objetivo de auxiliar empresas do ramo alimentício no gerenciamento do ciclo de vida de seus produtos, centralizando informações importantes em uma única plataforma.

Por meio do sistema é possível controlar desde a criação de uma receita até o lançamento do produto, registrando suas diferentes versões, os testes realizados, o histórico de alterações e o controle de estoque.

Além disso, o sistema busca reduzir a perda de documentos relacionados ao desenvolvimento dos produtos, organizar o processo de inovação da empresa e fornecer maior rastreabilidade das atividades realizadas pelos usuários.

A aplicação foi desenvolvida utilizando arquitetura baseada em API REST, empregando o framework Spring Boot no back-end, banco de dados MySQL e interface desenvolvida utilizando HTML, CSS e JavaScript.

2.2 TECNOLOGIAS UTILIZADAS

O desenvolvimento do sistema envolveu diversas tecnologias que, em conjunto, proporcionam segurança, desempenho, organização e facilidade de manutenção.

Tecnologia	Finalidade
Java 21	Linguagem utilizada para o desenvolvimento do back-end da aplicação.
Spring Boot	Framework responsável pela construção da API REST.
Spring Data JPA	Responsável pela persistência dos dados e comunicação entre a aplicação e o banco de dados.
Maven	Gerenciador de dependências e automação da compilação do projeto.
MySQL Server	Sistema Gerenciador de Banco de Dados utilizado pela aplicação.
MySQL Workbench	Ferramenta utilizada para administração e modelagem do banco de dados.
IntelliJ IDEA	Ambiente de Desenvolvimento Integrado (IDE) utilizado no desenvolvimento do back-end.
Visual Studio Code	Ambiente utilizado para o desenvolvimento do frontend.
Git	Sistema de controle de versão utilizado durante o desenvolvimento.
GitHub	Plataforma utilizada para hospedagem e versionamento do código-fonte.
Postman	Ferramenta utilizada para testes dos endpoints da API REST.

Railway	Plataforma utilizada para hospedagem da API e do banco de dados MySQL
---------	---

2.3 REQUISITOS DO SISTEMA

Para executar corretamente o sistema, é necessário que o computador possua os seguintes requisitos.

Requisitos de Hardware:

- Processador Dual Core ou superior;
- Memória RAM de, no mínimo, 4 GB;
- Espaço livre em disco de pelo menos 2 GB.

Requisitos de Software:

- Windows 10 ou superior;
- Java Development Kit (JDK) 21;
- Apache Maven;
- IntelliJ IDEA;
- MySQL Server;
- MySQL Workbench;
- Visual Studio Code;
- Git;
- Google Chrome ou outro navegador atualizado.

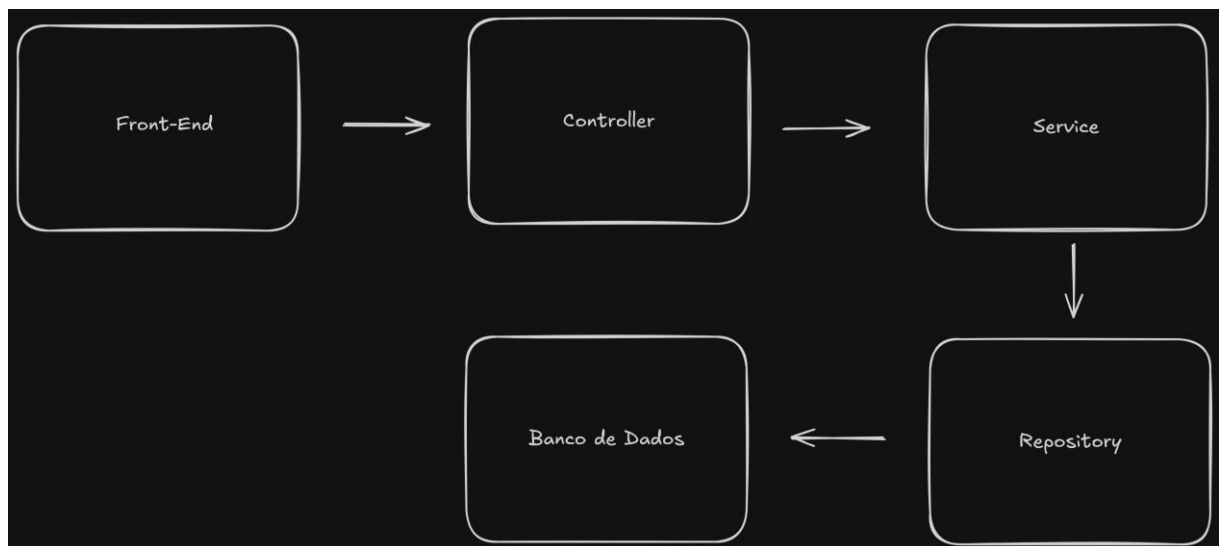
2.4 ESTRUTURA DO PROJETO (BACK-END)

O back-end do sistema foi desenvolvido utilizando uma arquitetura em camadas, padrão amplamente utilizado no desenvolvimento de aplicações web.

Essa arquitetura organiza a aplicação em diferentes níveis de responsabilidade, tornando o código mais limpo, organizado e de fácil manutenção. Cada camada possui funções específicas e se comunica apenas com as camadas necessárias para o funcionamento da aplicação.

O fluxo básico de funcionamento do sistema ocorre conforme ilustrado abaixo.

Figura 1- Arquitetura em camadas do sistema



Fonte: Elaborado pelos autores (2026)

2.4.1 CAMADA CONTROLLER

A camada **Controller** é responsável por receber as requisições HTTP enviadas pelo cliente.

Nela são definidos os endpoints da API REST, responsáveis por disponibilizar as funcionalidades do sistema. Após receber uma requisição, o Controller encaminha os dados para a camada **Service**, responsável pelo processamento das regras de negócio, retornando posteriormente uma resposta ao cliente.

Sua principal responsabilidade é realizar a comunicação entre o cliente e a aplicação.

2.4.2 CAMADA DTO

A camada **DTO (Data Transfer Object)** é responsável por controlar quais informações serão enviadas e recebidas pela API.

Sua utilização evita que dados desnecessários ou sensíveis presentes nas entidades sejam expostos ao cliente, permitindo maior controle sobre as informações trafegadas entre o back-end e o front-end.

Neste projeto, foram implementados DTOs para as entidades **Usuário, Produto, Estoque, Receita, VersãoReceita, Teste e Histórico**, permitindo que apenas os dados necessários sejam disponibilizados nas requisições e respostas da API.

As entidades **Perfil e StatusProduto** não possuem DTO, pois são compostas apenas pelos atributos **id e nome**, considerados suficientes para utilização direta pela aplicação, não havendo necessidade de criar objetos específicos para transferência desses dados.

2.4.3 CAMADA EXCEPTION

A camada **Exception** é responsável pelo tratamento das exceções geradas durante a execução da aplicação.

Seu objetivo é centralizar o gerenciamento dos erros, permitindo que todas as exceções sejam tratadas de maneira padronizada. Essa abordagem evita a repetição de código nas demais camadas da aplicação e fornece respostas mais claras ao cliente sempre que uma operação não puder ser concluída.

Durante o desenvolvimento do sistema foram criadas exceções personalizadas para representar situações específicas de negócio, como registros não encontrados ou operações inválidas. Além disso, foi implementado um **Global Exception Handler**, responsável por interceptar essas exceções e retornar respostas HTTP apropriadas, acompanhadas de mensagens descritivas que facilitam a identificação do problema.

Essa estratégia torna a API mais organizada, melhora a experiência do desenvolvedor que irá consumi-la e facilita futuras manutenções.

2.4.4 CAMADA MODEL

A camada **Model** representa as entidades do sistema e estabelece o mapeamento entre os objetos Java e as tabelas do banco de dados.

Cada classe dessa camada corresponde a uma tabela existente no banco de dados, contendo seus respectivos atributos, relacionamentos e restrições definidos por meio das anotações do **Spring Data JPA**.

As entidades modeladas no projeto representam os principais componentes da aplicação, como usuários, produtos, estoque, receitas, versões de receitas, testes realizados, histórico de alterações, perfis de usuários e status dos produtos.

Essa camada é utilizada em praticamente toda a aplicação, servindo como base para o armazenamento, consulta e manipulação das informações persistidas no banco de dados.

2.4.5 CAMADA REPOSITORY

A camada **Repository** é responsável pela comunicação entre a aplicação e o banco de dados.

Por meio do **Spring Data JPA**, os repositórios disponibilizam métodos responsáveis pelas operações de cadastro, consulta, atualização e exclusão de registros, reduzindo significativamente a necessidade de implementação manual de comandos SQL.

Cada entidade do sistema possui um repositório específico, permitindo que as operações de persistência sejam realizadas de forma organizada e desacoplada das regras de negócio implementadas na camada Service.

Essa separação contribui para uma arquitetura mais limpa, facilitando tanto a manutenção quanto a reutilização do código.

2.4.6 CAMADA SERVICE

A camada **Service** concentra toda a lógica de negócio da aplicação.

É nessa camada que são realizadas as validações dos dados recebidos, a aplicação das regras definidas pelo sistema e a comunicação com os repositórios responsáveis pelo acesso ao banco de dados.

Sempre que uma requisição é recebida pelo Controller, ela é encaminhada para a camada Service, onde são verificadas condições como existência de registros,

validação das informações fornecidas e consistência dos dados antes que qualquer operação seja executada.

Além disso, essa camada também é responsável por lançar exceções quando alguma regra de negócio não é atendida, permitindo que o tratamento seja realizado pela camada Exception e que a API retorne respostas padronizadas ao cliente.

Sua utilização mantém o Controller mais simples e concentra toda a lógica da aplicação em um único local, tornando o projeto mais organizado, reutilizável e de fácil manutenção.

2.4.7 CAMADA CONFIG

A camada **Config** é responsável pela configuração geral da aplicação, concentrando definições técnicas e comportamentos que serão utilizados pelo sistema como um todo. É nela que são definidos componentes essenciais do Spring, como configurações de segurança, CORS, beans personalizados e outras configurações globais que controlam o funcionamento da aplicação.

Essa camada não contém regras de negócio, mas sim regras estruturais e de infraestrutura, garantindo que o sistema funcione corretamente e de forma padronizada. Por exemplo, nela podem ser configurados filtros de segurança, codificação de senhas, configuração de autenticação, integração com bibliotecas externas e definição de políticas de acesso à API.

Sempre que o Spring precisa de um comportamento personalizado ou de um objeto gerenciado globalmente, ele é declarado na camada **Config** através de anotações como `@Configuration` e `@Bean`. Esses componentes são então injetados automaticamente em outras partes da aplicação, como **Services** e **Controllers**.

Sua utilização torna o projeto mais organizado e modular, separando claramente a configuração do sistema da lógica de negócio, o que facilita a manutenção, escalabilidade e entendimento da aplicação.

2.5 BANCO DE DADOS

O sistema utiliza o MySQL como sistema Gerenciador de Banco de Dados (SGBD), sendo responsável pelo armazenamento permanente de todas as informações utilizadas pela aplicação.

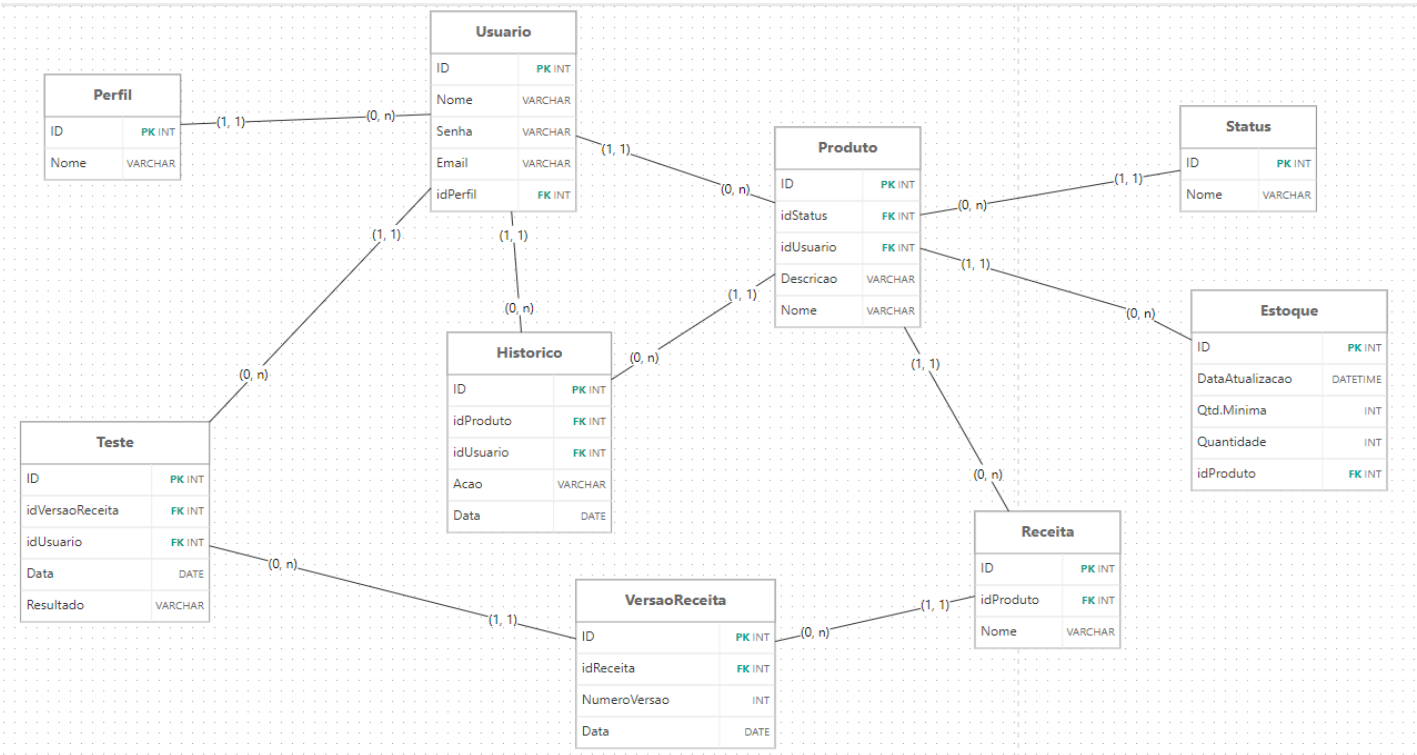
A escolha do MySQL ocorreu devido à sua confiabilidade, desempenho, facilidade de integração com o framework Spring Boot e ampla utilização no desenvolvimento de aplicações web.

A comunicação entre a aplicação e o banco de dados é realizada por meio do **Spring Data JPA**, que utiliza o conceito de Mapeamento Objeto-Relacional (Object-Relational Mapping - ORM). Essa abordagem permite que as entidades Java sejam automaticamente associadas às tabelas do banco de dados, simplificando as operações de persistência e reduzindo a necessidade da escrita manual de comandos SQL.

O banco de dados foi modelado visando garantir a integridade das informações, minimizar redundâncias e facilitar futuras manutenções no sistema. Sua estrutura contempla entidades responsáveis pelo gerenciamento de usuários, produtos, estoque, receitas, versões de receitas, testes, histórico de alterações e perfis de acesso.

A **Figura 2** apresenta o Diagrama Entidade-Relacionamento (DER) do banco de dados desenvolvido para o sistema.

Figura 2 – Diagrama Entidade-Relacionamento do banco de dados



Fonte: Elaborado pelos autores (2026)

2.5.1 TABELA PERFIL

A tabela **Perfil** é responsável por armazenar os diferentes perfis de acesso existentes no sistema.

Cada perfil representa um nível de permissão que poderá ser associado aos usuários cadastrados, permitindo controlar as funcionalidades disponíveis para cada tipo de usuário.

Como essa tabela possui apenas informações básicas de identificação, não foi necessária a implementação de um DTO específico para sua manipulação.

Campo	Tipo	Descrição
id	INT	Identificador único do perfil.
nome	VARCHAR(100)	Nome do perfil de acesso.

2.5.2 TABELA USUÁRIO

A tabela **Usuário** armazena os dados dos usuários responsáveis pela utilização do sistema.

Além das informações de identificação, cada usuário está obrigatoriamente vinculado a um perfil de acesso, permitindo que o sistema identifique suas permissões durante a utilização da aplicação.

O endereço de e-mail é único para cada usuário, impedindo cadastros duplicados.

Campo	Tipo	Descrição
id	INT	Identificador único do usuário.
nome	VARCHAR(100)	Nome do usuário.
senha	VARCHAR(255)	Senha do usuário.
email	VARCHAR(100)	Endereço de e-mail utilizado pelo usuário.
IdPerfil	INT	Perfil associado ao usuário.

2.5.3 TABELA STATUS_VERSAO_PRODUTO

A tabela **Status_Versao_Produto** armazena os possíveis estados em que uma versão da receita pode se encontrar durante seu ciclo de desenvolvimento.

Essa separação evita a repetição de informações dentro da tabela de versão receita e facilita a inclusão de novos status caso seja necessário futuramente.

Assim como a tabela **Perfil**, essa entidade possui apenas informações básicas e, por esse motivo, não utiliza DTO.

Campo	Tipo	Descrição
id	INT	Identificador do status.
nome	VARCHAR(100)	Nome do status da versão.

2.5.4 TABELA PRODUTO

A tabela **Produto** representa os produtos cadastrados na aplicação.

Cada produto possui informações básicas de identificação, uma descrição detalhada, um status e um usuário responsável pelo seu cadastro.

Essa entidade serve como base para diversos outros módulos do sistema, sendo relacionada ao estoque, às receitas e ao histórico de alterações.

Campo	Tipo	Descrição
id	INT	Identificador do produto.
Nome	VARCHAR(100)	Nome do produto.
Categoria	VARCHAR(100)	Categoria do produto
descrição	TEXT	Descrição do produto.
idStatus	INT	Status atual do produto.
idUsuario	INT	Usuário responsável pelo produto.

2.5.5 TABELA ESTOQUE

A tabela **Estoque** é responsável pelo controle das quantidades disponíveis de cada produto.

Além da quantidade atual, são armazenadas a quantidade mínima permitida e a data da última atualização das informações.

Cada produto possui apenas um registro de estoque, caracterizando um relacionamento do tipo **um para um** (1:1).

Campo	Tipo	Descrição
id	INT	Identificador do estoque.
data_atualizacao	DATETIME	Data da última atualização do estoque.
lote	VARCHAR(100)	Lote do produto que está no estoque.
data_validade	DATE	Data de validade do lote.
data_fabricacao	DATE	Data de fabricação do lote.
qtd_minima	INT	Quantidade mínima do produto.
quantidade	INT	Quantidade disponível.
idProduto	INT	Produto associado ao estoque.

2.5.6 TABELA RECEITA

A tabela **Receita** armazena as receitas desenvolvidas para cada produto.

Cada receita pertence a um único produto, porém um produto pode possuir diversas receitas cadastradas ao longo do seu desenvolvimento.

Essa estrutura permite registrar diferentes formulações para um mesmo produto.

Campo	Tipo	Descrição
id	INT	Identificador da receita.
nome	VARCHAR(100)	Nome da receita.
idProduto	INT	Produto relacionado à receita.

2.5.7 TABELA VERSAO_RECEITA

A tabela **Versão_Receita** registra todas as versões criadas para cada receita.

Esse controle permite acompanhar a evolução da formulação dos produtos durante o processo de desenvolvimento.

Cada versão possui um número identificador e a data em que foi criada.

Campo	Tipo	Descrição
Id	INT	Identificador da versão.
numero_versao	INT	Número da versão.
data_versao	DATE	Data da versão.
idReceita	INT	Receita relacionada à versão.
idStatus	INT	Status relacionado à versão.

2.5.8 TABELA TESTE

A tabela **Teste** registra os testes realizados em cada versão da receita.

Cada registro contém a data do teste, seu resultado, o usuário responsável pela execução e a versão da receita utilizada durante a avaliação.

Essas informações permitem acompanhar a evolução do desenvolvimento do produto antes de sua aprovação.

Campo	Tipo	Descrição
id	INT	Identificador do teste.
data_teste	DATETIME	Data de realização do teste.
resultado	TEXT	Resultado obtido.
idUsuario	INT	Usuário responsável.
idVersao_receita	INT	Versão da receita testada.

2.5.9 TABELA HISTÓRICO

A tabela **Histórico** registra as principais ações realizadas pelos usuários durante a utilização do sistema.

Cada registro armazena a descrição da ação executada, sua data e hora, o usuário responsável e o produto relacionado.

Esse mecanismo permite rastrear alterações importantes realizadas na aplicação.

Campo	Tipo	Descrição
id	INT	Identificador do registro.
acao	VARCHAR(200)	Descrição da ação realizada.
data_historico	DATETIME	Data e hora da ação.
idProduto	INT	Produto relacionado ao histórico.
idUsuario	INT	Usuário relacionado ao histórico.

2.5.10 RELACIONAMENTOS

O banco de dados foi estruturado utilizando chaves primárias e estrangeiras para garantir a integridade referencial entre as tabelas.

Os principais relacionamentos implementados são:

- Um **Perfil** pode estar associado a vários **Usuários** (1:N).
- Um **Usuário** pertence a apenas um **Perfil** (N:1).
- Um **Status da Versão** pode estar associado a diversas **Versões** (1:N).
- Um **Usuário** pode cadastrar diversos **Produtos** (1:N).
- Um **Produto** possui apenas um **Estoque** (1:1).
- Um **Produto** pode possuir diversas **Receitas** (1:N).
- Uma **Receita** pode possuir diversas **Versões** (1:N).
- Uma **Versão da Receita** pode possuir diversos **Testes** (1:N).
- Um **Usuário** pode realizar diversos **Testes** (1:N).
- Um **Produto** pode possuir diversos registros no **Histórico** (1:N).

- Um **Usuário** pode gerar diversos registros no **Histórico** (1:N).

2.6 ESTRUTURA DA API

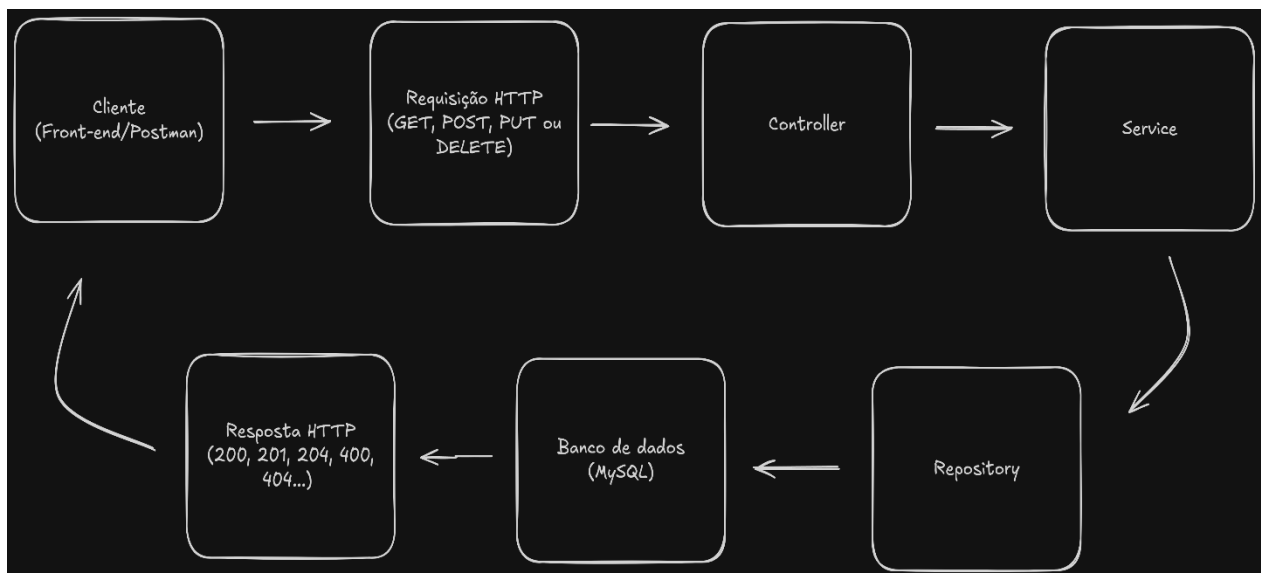
A API do sistema **OneMonth** foi desenvolvida seguindo o padrão arquitetural **REST (Representational State Transfer)**, amplamente utilizado no desenvolvimento de aplicações web devido à sua simplicidade, organização e facilidade de integração entre diferentes sistemas.

A comunicação entre o front-end e o back-end é realizada por meio do protocolo **HTTP**, utilizando diferentes métodos para representar as operações realizadas sobre os recursos da aplicação.

Cada funcionalidade do sistema é organizada em um **Controller**, responsável por disponibilizar os endpoints da API relacionados a uma determinada entidade. Dessa forma, cada controller possui a responsabilidade de receber as requisições, encaminhá-las para a camada Service e retornar uma resposta ao cliente.

A Figura 3 apresenta o fluxo simplificado de uma requisição realizada à API.

Figura 3 – Fluxo de uma requisição na API



Fonte: Elaborado pelos autores (2026)

Conforme ilustrado na Figura 3, uma requisição enviada pelo cliente é inicialmente recebida pela camada **Controller**, que encaminha as informações para a camada **Service**. Após a aplicação das regras de negócio, a camada **Repository** realiza a comunicação com o banco de dados, retornando o resultado da operação até o cliente.

2.6.1 MÉTODOS HTTP

A API utiliza os principais métodos do protocolo HTTP para representar as operações realizadas sobre os recursos do sistema.

Método	Finalidade
GET	Realiza consulta de informações.
POST	Realiza o cadastro de novos registros.
PUT	Atualiza informações existentes.
DELETE	Remove registros do sistema.

A utilização desses métodos segue o padrão REST, permitindo que as operações sejam realizadas de maneira padronizada e intuitiva.

2.6.2 USUARIO CONTROLLER

O **UsuarioController** concentra os endpoints responsáveis pelo gerenciamento dos usuários do sistema.

Por meio dele é possível realizar operações de cadastro, consulta, atualização e exclusão de usuários.

Método HTTP	Endpoint	Descrição
GET	/usuario	Lista todos os usuários cadastrados.
GET	/usuario/{id}	Retorna um usuário específico.
POST	/usuario	Realiza o cadastro de um novo usuário.

PUT	/usuario/{id}	Atualiza as informações de um usuário.
DELETE	/usuario/{id}	Remove um usuário do sistema.

2.6.3 PERFIL CONTROLLER

O **PerfilController** é responsável pelo gerenciamento dos perfis de acesso cadastrados no sistema.

Por meio desse controller é possível consultar, cadastrar, atualizar e remover perfis de usuários, permitindo o gerenciamento dos diferentes níveis de acesso existentes na aplicação.

Método HTTP	Endpoint	Descrição
GET	/perfil	Lista todos os perfis cadastrados.
GET	/perfil/{id}	Consulta um perfil específico.
POST	/perfil	Cadastra um novo perfil.
PUT	/perfil/{id}	Atualiza um perfil existente.
DELETE	/perfil/{id}	Remove um perfil.

2.6.4 PRODUTO CONTROLLER

O **ProdutoController** é responsável pelo gerenciamento dos produtos cadastrados.

Os endpoints disponibilizados permitem cadastrar novos produtos, consultar registros existentes, realizar alterações e remover produtos do sistema.

Método HTTP	Endpoint	Descrição
GET	/produtos	Lista todos os produtos cadastrados.

GET	/produtos/id{id}	Retorna um produto específico.
POST	/produtos	Realiza o cadastro de um novo produto.
PUT	/produtos/{id}	Atualiza as informações de um produto.
DELETE	/produtos/id{id}	Remove um produto do sistema.

2.6.5 StatusProduto CONTROLLER

O **StatusProdutoController** concentra os endpoints responsáveis pelo gerenciamento dos status dos produtos.

Esses status representam as diferentes etapas do ciclo de desenvolvimento dos produtos, permitindo sua organização durante todo o processo.

Método HTTP	Endpoint	Descrição
GET	/status	Lista todos os status cadastrados.
GET	/status/{id}	Consulta um status específico.
POST	/status	Cadastra um novo status.
PUT	/status/{id}	Atualiza um status existente.
DELETE	/status/{id}	Remove um status.

2.6.6 ESTOQUE CONTROLLER

O **EstoqueController** é responsável pelas operações relacionadas ao controle de estoque dos produtos.

Método HTTP	Endpoint	Descrição
-------------	----------	-----------

GET	/estoque	Lista os registros de estoque.
GET	/estoque/{id}	Consulta um registro específico.
POST	/estoque	Cadastra um novo registro de estoque.
PUT	/estoque/{id}	Atualiza as informações do estoque.
DELETE	/estoque/{id}	Remove um registro de estoque.

2.6.7 RECEITA CONTROLLER

O **ReceitaController** disponibiliza os endpoints responsáveis pelo gerenciamento das receitas cadastradas para os produtos.

Método HTTP	Endpoint	Descrição
GET	/receita	Lista todas as receitas.
GET	/receita/{id}	Consulta uma receita específica.
POST	/receita	Cadastra uma nova receita.
PUT	/receita/{id}	Atualiza uma receita existente.
DELETE	/receita/{id}	Remove uma receita.

2.6.8 VersaoReceita CONTROLLER

O **VersaoReceitaController** é responsável pelo gerenciamento das diferentes versões das receitas cadastradas no sistema.

Método HTTP	Endpoint	Descrição
-------------	----------	-----------

GET	/versaoreceita	Lista todas as versões.
GET	/versaoreceita/{id}	Consulta uma versão específica.
POST	/versaoreceita	Cadastra uma nova versão.
PUT	/versaoreceita/{id}	Atualiza uma versão existente.
DELETE	/versaoreceita/{id}	Remove uma versão.

2.6.9 TESTE CONTROLLER

O **TesteController** gerencia os testes realizados nas versões das receitas.

Método HTTP	Endpoint	Descrição
GET	/teste	Lista todos os testes.
GET	/teste/{id}	Consulta um teste específico.
POST	/teste	Registra um novo teste.
PUT	/teste/{id}	Atualiza um teste existente.
DELETE	/teste/{id}	Remove um teste.

2.6.10 HISTORICO CONTROLLER

O **HistoricoController** disponibiliza os endpoints responsáveis pelo registro e consulta do histórico de alterações realizadas no sistema.

Método HTTP	Endpoint	Descrição
GET	/historico	Lista o histórico de alterações.
GET	/historico/{id}	Consulta um registro específico.
POST	/historico	Registra uma nova ação no histórico.
PUT	/historico/{id}	Atualiza um registro no histórico.
DELETE	/historico/{id}	Remove um registro do histórico.

2.6.11 AUTH CONTROLLER

O **AuthController** disponibiliza apenas o endpoint **POST**, responsável por fazer o login com os dados enviados pelo frontend.

Método HTTP	Endpoint	Descrição
POST	/auth/login	Realiza o login com os dados enviados pelo frontend.

2.7 TRATAMENTO DE EXCEÇÕES

O tratamento de exceções do sistema **OneMonth** foi implementado com o objetivo de centralizar o gerenciamento de erros e padronizar as respostas retornadas pela API.

Quando ocorre uma situação inesperada durante o processamento de uma requisição, como a tentativa de acessar um recurso inexistente ou o envio de dados inválidos, a aplicação captura a exceção correspondente e retorna uma resposta estruturada em formato JSON, contendo informações sobre o erro ocorrido.

Essa abordagem melhora a comunicação entre o back-end e o front-end, facilita a identificação de problemas e contribui para a organização e manutenção do código.

2.7.1 GlobalExceptionHandler

A classe **GlobalExceptionHandler** é responsável por interceptar automaticamente as exceções lançadas durante a execução da aplicação.

Sua implementação utiliza a anotação `@RestControllerAdvice`, disponibilizada pelo Spring Boot, permitindo que todas as exceções tratadas sejam centralizadas em uma única classe.

Cada método anotado com `@ExceptionHandler` é responsável por capturar um tipo específico de exceção e construir uma resposta HTTP padronizada utilizando a classe **ErrorResponse**.

Essa estratégia evita a repetição de código nos controllers e services, além de tornar a API mais consistente e de fácil manutenção.

2.7.2 ErrorResponse

A classe **ErrorResponse** define o modelo utilizado para retornar informações sobre erros ocorridos durante a execução da API.

Cada resposta contém três informações principais:

Campo	Descrição
timestamp	Data e hora em que ocorreu o erro.
tatus	Código HTTP retornado pela API.
mensagem	Descrição do erro ocorrido.

2.7.3 EXCEÇÕES PERSONALIZADAS

Com o objetivo de representar situações específicas de erro, foram implementadas exceções personalizadas que estendem a classe `RuntimeException`.

Essas exceções permitem identificar com maior precisão o tipo de problema ocorrido durante o processamento das requisições.

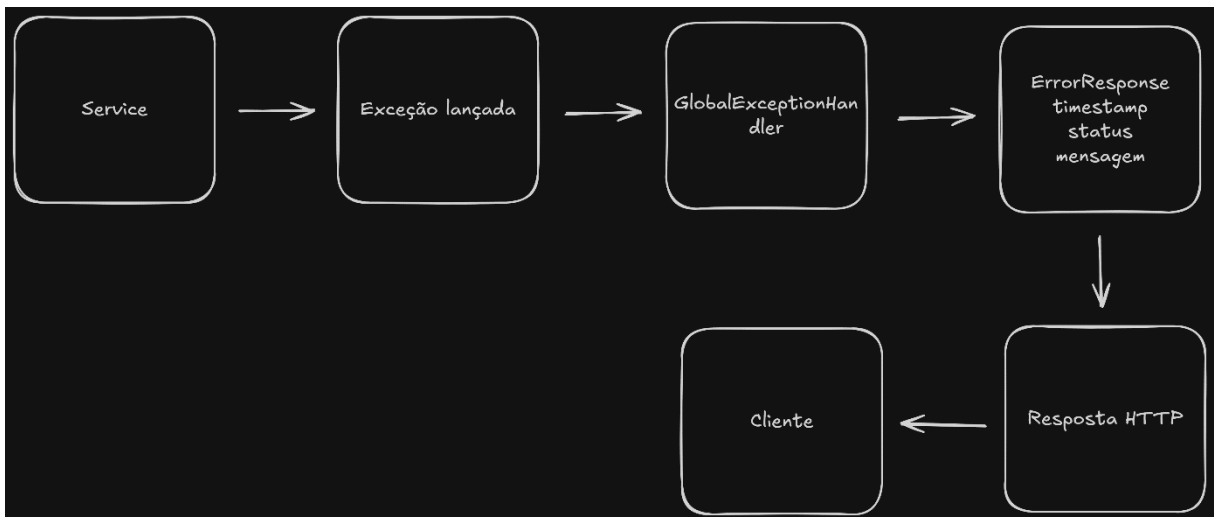
Exceção	Código HTTP	Finalidade
ResourceNotFoundException	404 (Not Found)	Lançada quando o recurso solicitado não é encontrado no banco de dados.
ValidationException	400 (Bad Request)	Lançada quando os dados enviados na requisição não atendem às regras de validação da aplicação.
EmailJaCadastradoException	400(Bad Request)	Lançada quando o email enviado pelo frontend no cadastro já existe no banco.
CredenciaisInvalidasException	401(Unauthorized)	Lançado quando os dados enviados pelo frontend no login não coincidem com nenhum salvos no banco.

2.7.4 FLUXO DO TRATAMENTO DE EXCEÇÕES

Durante o processamento de uma requisição, caso uma exceção seja lançada pela camada **Service**, ela é interceptada pelo **GlobalExceptionHandler**, que cria um objeto **ErrorResponse** contendo as informações do erro e o retorna ao cliente juntamente com o código HTTP correspondente.

Esse fluxo garante que todas as respostas de erro possuam o mesmo formato, independentemente da origem da exceção.

Figura 4 – Fluxo do tratamento de exceções



Fonte: Elaborado pelos autores (2026)

2.7.5 EXEMPLO DE RESPOSTA

Quando ocorre uma exceção, a API retorna uma resposta em formato JSON semelhante ao exemplo abaixo:

```
{  
  "timestamp": "2026-07-01T20:15:43.512",  
  "status": 404,  
  "mensagem": "Teste não encontrado!"  
}
```

3. MANUAL DE INSTALAÇÃO E EXECUÇÃO (BACKEND)

3.1 OBJETIVO

Este capítulo apresenta os procedimentos necessários para configurar o ambiente de desenvolvimento, executar o back-end do sistema OneMonth e realizar testes na API REST.

Além disso, são descritas as ferramentas utilizadas durante o desenvolvimento e a forma de acesso ao banco de dados hospedado na plataforma Railway.

3.2 REQUISITOS DO AMBIENTE

Para executar o back-end do sistema é necessário possuir as seguintes ferramentas instaladas:

Software	Versão Recomendada	Finalidade
Java JDK	21	Execução da aplicação
IntelliJ IDEA	Community ou Ultimate	Desenvolvimento do projeto
Maven	3.9 ou superior	Gerenciamento de depêndencias
Git	Última versão	Clonagem do projeto
Postman	Última versão	Testes da API
MySQL workbench	8.0 ou superior	Administração do banco de dados

3.3 CLONANDO O PROJETO

O código-fonte do back-end encontra-se hospedado em um repositório Git.

Para obter o projeto, execute o seguinte comando:

```
“git clone URL_DO_REPOSITORIO”
```

Após a clonagem, abra o projeto utilizando o IntelliJ IDEA.

Ao abrir o projeto pela primeira vez, o Maven realizará automaticamente o download de todas as dependências definidas no arquivo `pom.xml`.

3.4 CONFIGURAÇÃO DO BANCO DE DADOS

As configurações de conexão com o banco de dados encontram-se no arquivo:

```
“src/main/resources/application.properties”
```

O arquivo deve conter as informações de conexão com a instância MySQL hospedada no Railway.

Exemplo:

```
“spring.datasource.url=${DATABASE_URL}
spring.datasource.username=${DATABASE_USERNAME}
spring.datasource.password=${DATABASE_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true”
```

No projeto, as credenciais do banco de dados são armazenadas em **variáveis de ambiente**, evitando que informações sensíveis fiquem expostas no código-fonte.

3.6 EXECUÇÃO DA APLICAÇÃO

Após abrir o projeto no IntelliJ IDEA e realizar o download das dependências do Maven, execute a classe principal da aplicação.

Durante a inicialização, o Spring Boot realizará automaticamente:

- carregamento das dependências;
- configuração da aplicação;
- conexão com o banco de dados hospedado no Railway;
- inicialização da API REST.

Quando a inicialização for concluída, o console apresentará mensagens indicando que a aplicação foi iniciada com sucesso.

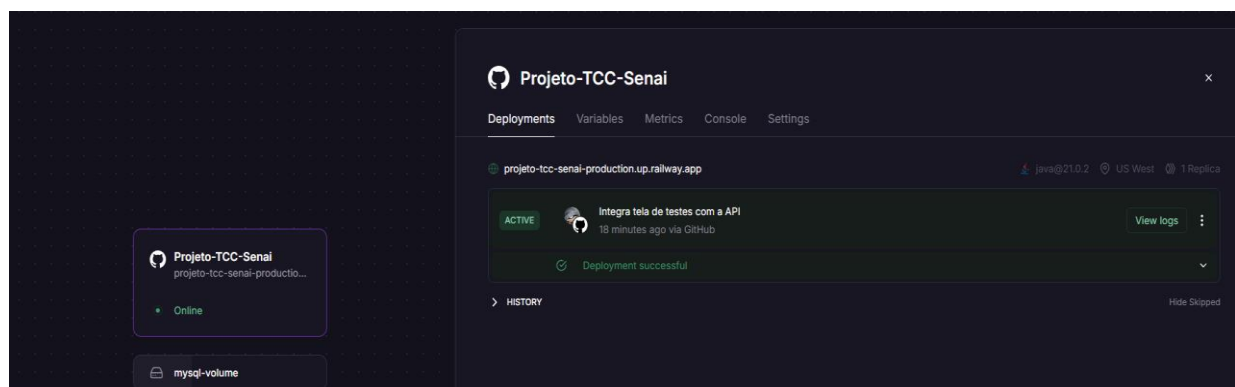
3.7 HOSPEDAGEM DA API

Após os testes locais, o back-end foi publicado na plataforma **Railway**, permitindo que a API fique disponível para acesso remoto.

A hospedagem da API possibilita que o front-end realize requisições HTTP sem a necessidade de executar o servidor localmente.

Sempre que uma nova versão do projeto é enviada ao repositório, a plataforma Railway realiza automaticamente uma nova implantação da aplicação.

Figura 5 – API hospedada no railway



Fonte: Elaborado pelos autores(2026)

3.8 TESTANDO A API

Os testes da API podem ser realizados utilizando o software **Postman**.

Após iniciar a aplicação localmente ou utilizar a versão hospedada no Railway, basta enviar requisições HTTP para os endpoints disponíveis.

Exemplo:

“GET /produtos”

Resposta

esperada:

```
“  
  
{  
  "id": 1,  
  "nome": "Produto Exemplo"  
}  
”
```

Também podem ser realizados testes utilizando os métodos:

- GET
- POST
- PUT
- DELETE

verificando os códigos HTTP retornados e as mensagens de erro quando necessário.

3.9 SEGURANÇA DAS SENHAS

Para aumentar a segurança das informações armazenadas no sistema, as senhas dos usuários são criptografadas utilizando o algoritmo **BCrypt**.

Durante o cadastro de um usuário, a senha informada não é armazenada em texto puro no banco de dados. Antes da gravação, ela é convertida para um hash criptográfico por meio da biblioteca BCrypt.

No momento do login, a senha informada pelo usuário é comparada com o hash armazenado no banco de dados, garantindo a autenticação sem que a senha original seja exposta.

Essa abordagem aumenta significativamente a segurança da aplicação, reduzindo os riscos associados ao vazamento de credenciais.

3.10 SOLUÇÃO DE PROBLEMAS

Durante a execução da aplicação podem ocorrer algumas situações comuns.

Problema	Possível solução
Erro de conexão com o banco	Verificar as credenciais do Railway e as variáveis de ambiente.
Maven não baixa as dependências	Atualizar o projeto Maven no IntelliJ IDEA.
Porta em uso	Alterar a porta da aplicação ou finalizar o processo que está utilizando a porta configurada.
API não inicia	Verificar as mensagens apresentadas no console do Spring Boot.